



Instituto Infnet

Análise e Segurança de Agentes de IA: Projeto de Bloco

Etapa 3 e 4

Professor: Tiago Cariolano de Souza Xavier

Email : tiago.xavier@prof.infnet.edu.br

OWASP Top 10



OWASP Top 10

- OWASP — Open Worldwide Application Security Project: mantém projetos e materiais sobre segurança de aplicações
- OWASP Top 10: lista de 10 categorias de riscos considerados especialmente relevantes em aplicações web
 - Não deve ser entendido como uma lista de dez ataques específicos
 - **Categorias:**
 - **A01 Broken Access Control**
 - A02 Security Misconfiguration
 - **A03 Software Supply Chain Failures**
 - A04 Cryptographic Failures
 - **A05 Injection**
 - A06 Insecure Design
 - A07 Authentication Failures
 - A08 Software or Data Integrity Failures
 - A09 Security Logging and Alerting Failures
 - A10 Mishandling of Exceptional Conditions



Entrada de campos não previstos

A01 Broken Access Control

Entrada de campos não previstos

- API define quais campos espera receber

- Exemplo:

- `from pydantic import BaseModel`

- ```
class PredictionRequest(BaseModel):
 text: str
```

- API espera somente:

- {

- `"text": "Meu pedido não chegou"`

- }



# Entrada de campos não previstos

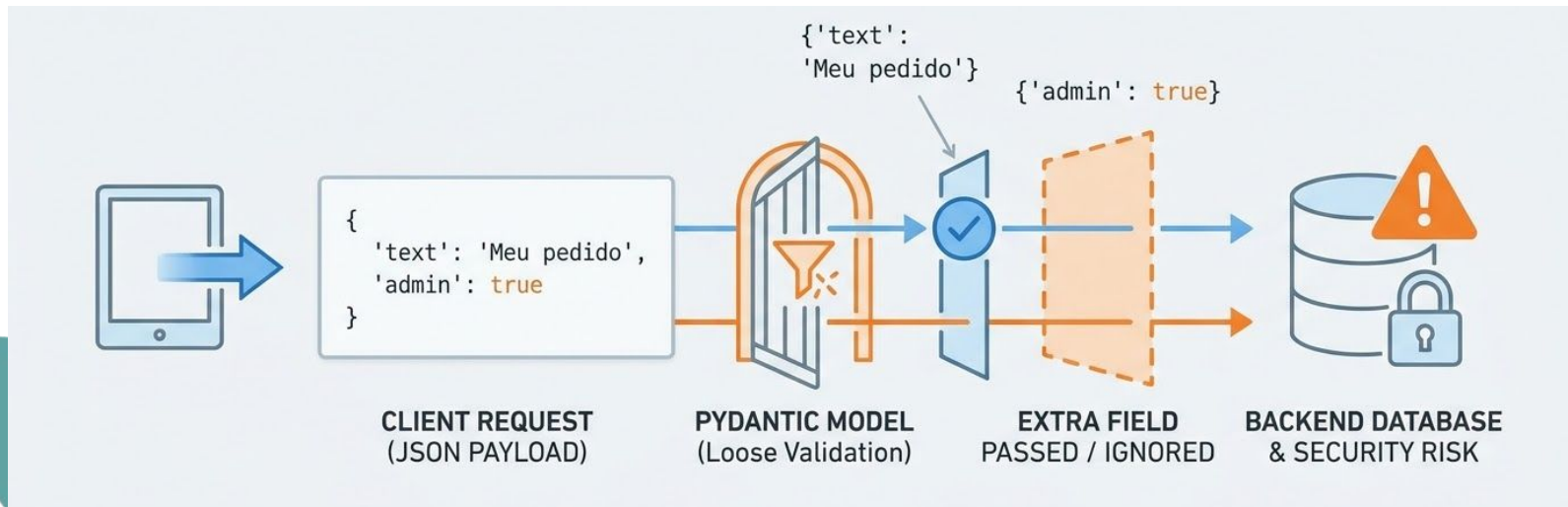
- Cliente controla o conteúdo da requisição e pode tentar enviar campos adicionais:

- ```
{  
    "text": "Meu pedido não chegou",  
    "admin": true,  
    "role": "administrator"  
}
```

- ***Se a aplicação não rejeitar explicitamente campos desconhecidos, dados que não fazem parte do contrato da API podem ser enviados e eventualmente utilizados de maneira indevida***

Entrada de campos não previstos

- Problema:



Entrada de campos não previstos

- Solução: configurar o parâmetro extra:forbid no modelo Pydantic:

- `from pydantic import BaseModel, ConfigDict`

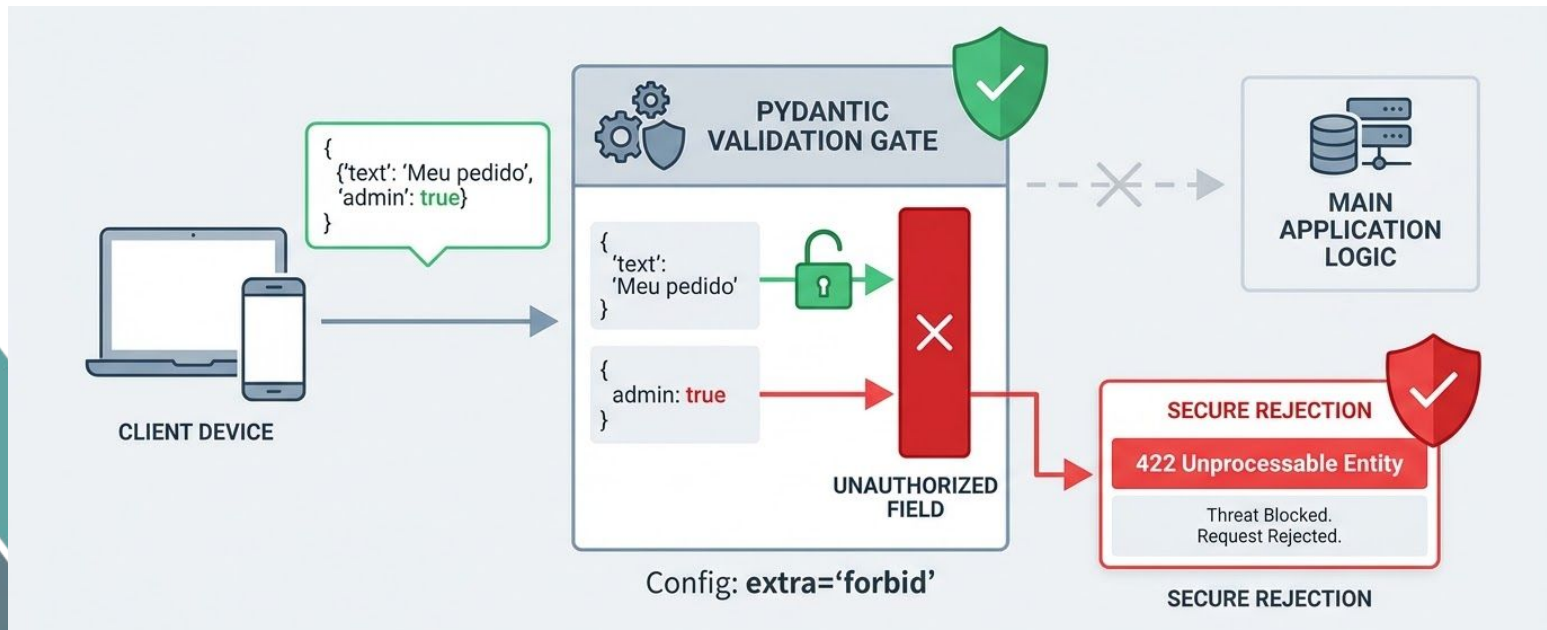
```
class PredictionRequest(BaseModel):  
    model_config = ConfigDict(extra="forbid")
```

```
text: str
```



Entrada de campos não previstos

- Solução:



SQL Injection

A05 Injection

Tutorial SQLite

- `conn = sqlite3.connect(database)`
 - `database`: Caminho do arquivo .db. Cria arquivo, caso não exista
 - `conn`: conexão ativa com o banco
- `cursor = conn.cursor()`
 - `cursor`: Objeto que permite realizar consultas SQL
- `cursor.execute(sql)`
 - `sql`: consulta SQL a ser executada no banco de dados
- `results = cursor.fetchall()`
 - Retorna lista de tuplas com as linhas da consulta realizada
- `conn.commit()`
 - Salva as operações realizadas
- `conn.close()`
 - Fecha a conexão

SQLite

- Exemplo: criar tabela user com id, username, password
 - `import sqlite3`
 - `conn = sqlite3.connect('database.db')`
 - `cursor = conn.cursor()`
 - `cursor.execute('''`
 - `CREATE TABLE IF NOT EXISTS user (`
 - `id INTEGER PRIMARY KEY AUTOINCREMENT,`
 - `username TEXT NOT NULL UNIQUE,`
 - `password TEXT NOT NULL)''')`
 - `conn.commit()`
 - `conn.close()`

SQLite

- **Exemplo: inserir dois usuários**

- `cursor.execute("INSERT INTO user (username, password) VALUES ('admin', 'admin')")`

- `cursor.execute("INSERT INTO user (username, password) VALUES ('jose', '1234')")`

- **Exemplo: listar todos os usuários:**

- `result = cursor.execute("SELECT * FROM user")`

- `for line in result:
 print(line)`

- **Exemplo: listar usuário pelo id:**

- `cursor.execute("SELECT * FROM user WHERE id = 1")
cursor.fetchall()`



SQL Injection

- Montagem manual de SQL pode gerar brechas de segurança

- Exemplo:

- ```
username = input("Username: ")
query = ("SELECT * FROM users " f"WHERE username = '{username}'")
```

- Cenário válido: usuário digitar joao

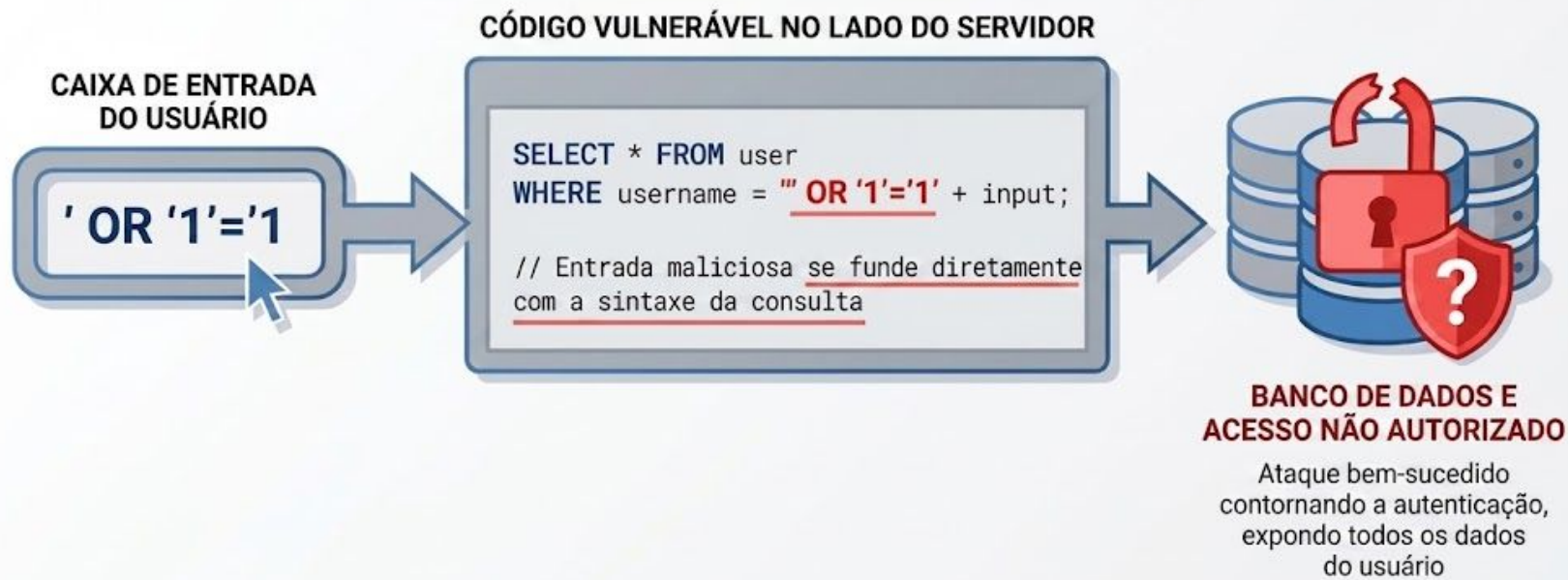
- ```
SELECT * FROM users WHERE username = 'joao'
```

- Cenário malicioso: atacante envia: ' OR '1'='1'

- ```
SELECT * FROM users WHERE username = '' OR '1'='1'
```



# SQL Injection



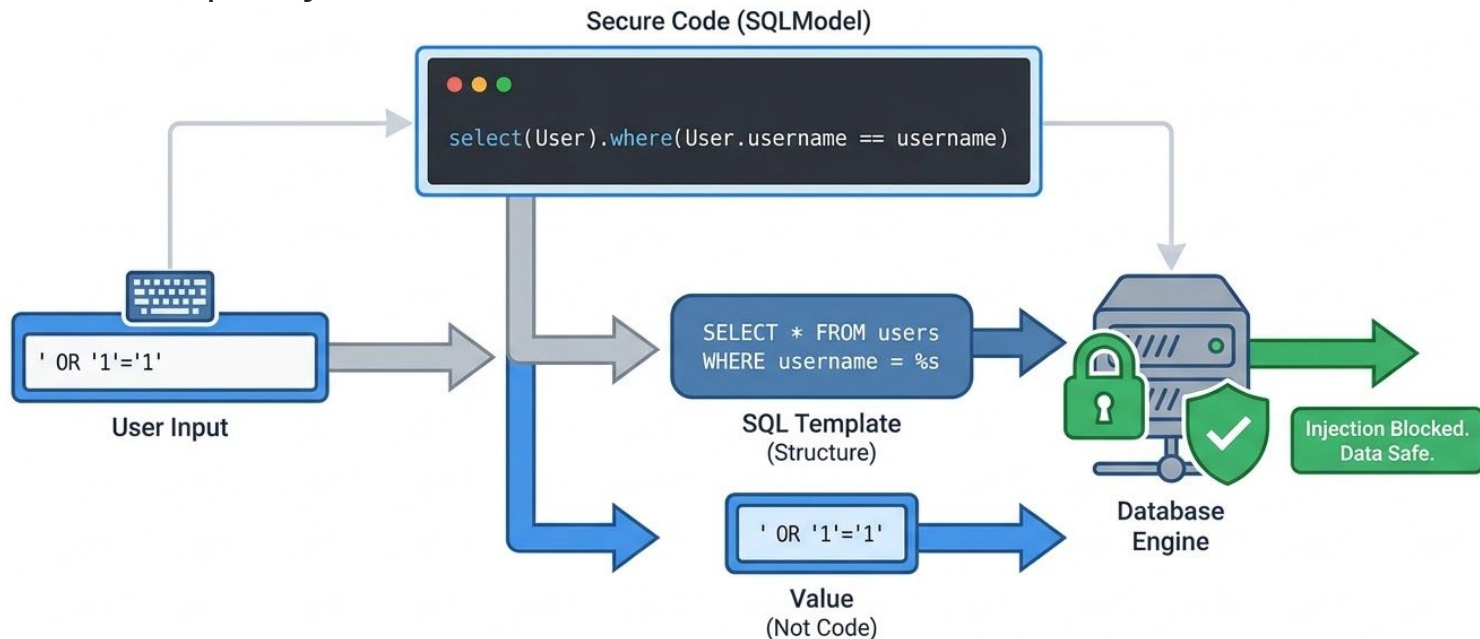
# SQLModel

- **SQLModel:** permite acessar tabelas do banco de dados utilizando classes e objetos Python, sem precisar escrever comandos SQL manualmente
- **Representação SQLModel**
  - Tabela **user** no BD com três colunas - **id, username, password**
  - SQLModel:

```
■ from sqlmodel import SQLModel, Field
 class User(SQLModel, table=True):
 id: int | None = Field(
 default=None,
 primary_key=True
)
 username: str
 password: str
```

# SQLModel

- SQLModel constrói a consulta
- Campos são tratados como valor, e não como código SQL
  - Separação entre COMANDO SQL e DADO



# SQLModel: exemplos

- **Buscar todos os usuários:**

- ```
from sqlmodel import Session, select

with Session(engine) as session:
    statement = select(User).where(User.username == 'joao')
    user = session.exec(statement).first()
```

- **Buscar usuário por ID:**

- ```
statement = select(User).where(User.id == user_id)
user = session.exec(statement).first()
```



# Verificação de Ownership

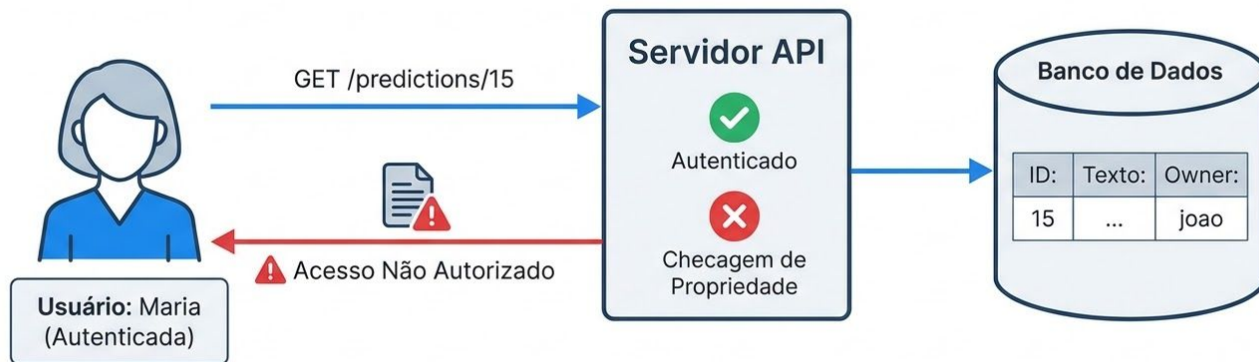
**A01 Broken Access Control**

# Vulnerabilidade BOLA

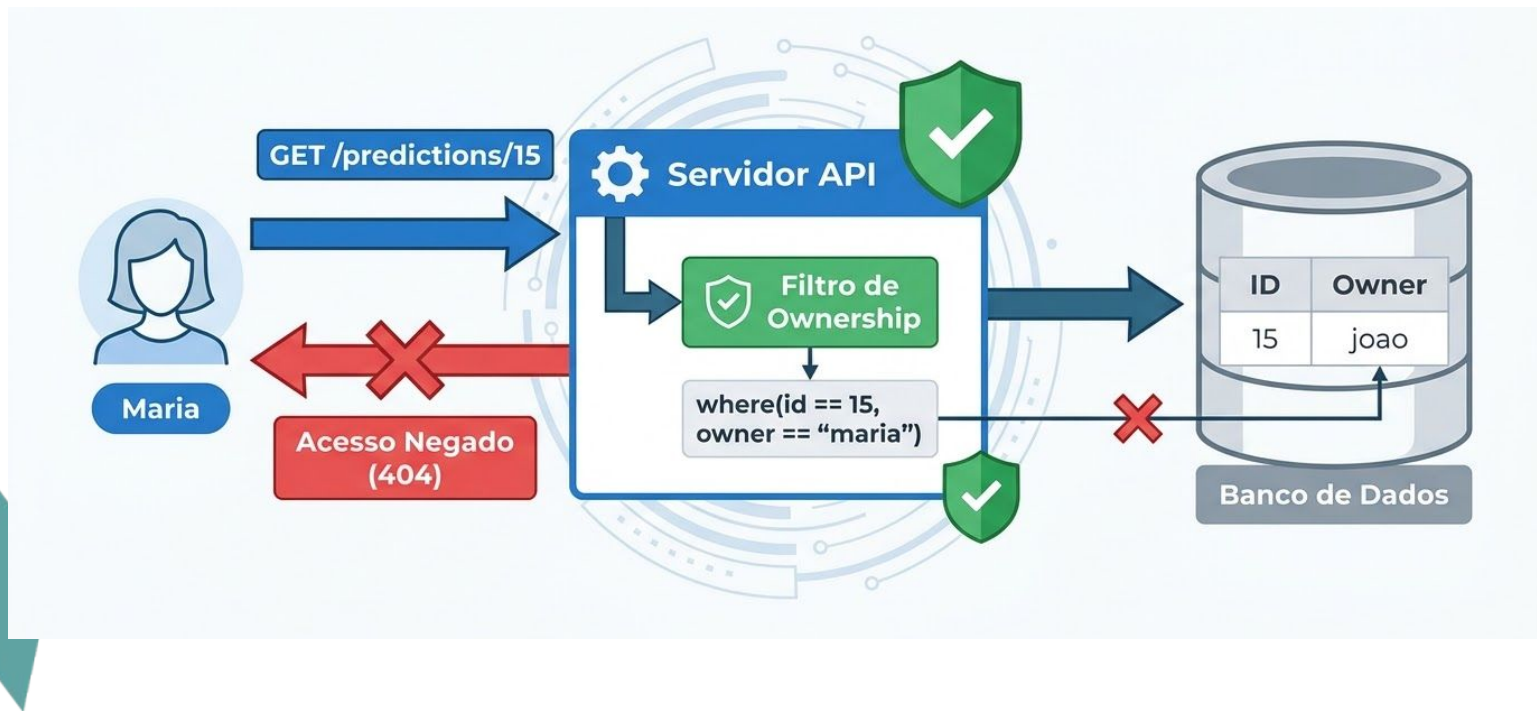
- **BOLA — Broken Object Level Authorization:** vulnerabilidade que ocorre quando a aplicação permite que um usuário acesse um objeto sem verificar se ele possui autorização sobre aquele objeto
- **Exemplo:**
  - **Premissas do sistema:**
    - Prediction 15 → owner = joao
    - Usuário autenticado → maria
  - **Se Maria conseguir acessar tem-se vulnerabilidade BOLA:**
    - GET /predictions/15
- **Autenticação/Autorização:**
  - **Autenticação:** processo utilizado para descobrir quem é o usuário
    - JWT
  - **Autorização:** processo utilizado para determinar o que o usuário pode acessar ou fazer
  - **Ownership:** quem é o proprietário de determinado recurso



# Vulnerabilidade BOLA



# Verificação de ownership



# Verificação de ownership para Predictions

- Tabelas

- prediction: id, text, intent, owner\_id
- user: id, username, password

- Modelos SQLAlchemy:

- ```
class User(SQLModel, table=True):  
    id: int | None = Field(default=None, unique=True, primary_key=True)  
    username: str  
    password: str
```

- ```
class Prediction(SQLModel, table=True):
 id: int | None = Field(default=None, primary_key=True)
 text: str
 intent: str
 owner_id: int = Field(foreign_key="user.id")
```

# Verificação de ownership para Predictions

- Tabelas

- prediction: id, text, intent, owner\_id
- user: id, username, password

- Modelos SQLAlchemy:

- ```
class User(SQLModel, table=True):  
    id: int | None = Field(default=None, unique=True, primary_key=True)  
    username: str  
    password: str
```

- ```
class Prediction(SQLModel, table=True):
 id: int | None = Field(default=None, primary_key=True)
 text: str
 intent: str
 owner_id: int = Field(foreign_key="user.id")
```

# Verificação de ownership para Predictions

- Módulo `dependencies.py`: armazena funções auxiliares
  - Função para obter usuário atual:

```
def get_current_user(username: str = Depends(validar_token), session: Session = Depends(get_session)):
 statement = select(User).where(User.username == username)
 user = session.exec(statement).first()
 if user is None:
 raise HTTPException(status_code=401, detail="Usuário não encontrado")
 return user
```

Função `validar_token` é a função implementada para validação jwt que recebe o token, decodifica ele e retorna o username

Função `get_session` é uma função auxiliar que cria uma sessão sqlite para acessar o banco de dados com SQLAlchemy.

# Verificação de ownership para Predictions

- Módulo auxiliar `database.py`:

- ```
from sqlalchemy import create_engine, Session
DATABASE_URL = "sqlite:///database.db"
engine = create_engine(DATABASE_URL)
def get_session():
    with Session(engine) as session:
        yield session
```



HTTP Security Headers

A02:2025 — Security
Misconfiguration

Headers HTTP

- **Header HTTP:** informação adicional (metadados de rede) enviadas em uma requisição ou resposta HTTP

- **Resposta HTTP:**

- HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 32

Headers (Cabeçalho)

- ```
{
 "mensagem": "Olá"
}
```

Paylload

- Headers informam o cliente ou servidor sobre informações da aplicação
  - **Incluem políticas de segurança**

# Middleware FastAPI

- **Middleware:** código que fica “no meio do caminho” entre a requisição que chega e a resposta que sai
  - **Fluxo:** Cliente→Middleware→Rota FastAPI→Middleware→Cliente
- **Middleware pode inspecionar ou modificar a requisição**
  - antes da rota ser gerada
  - ou depois da rota ser gerada
- **Ideal:** aplicar uma regra para toda a aplicação, sem repetir código
  - Exemplo: enviar quanto tempo o servidor demorou para processar a requisição
    - **Header X-Process-Time** em cada requisição
  - Sem middleware tem que repetir código em cada requisição



# Middleware FastAPI

- **Sintaxe:**
  - ```
@app.middleware("http")
async def nome_middleware(request: fastapi.Request,
call_next):
    # código executado antes da rota
    response = await call_next(request)
    # código executado depois da rota
    return response
```

 - **request:** requisição que chegou
 - **call_next:** função que continua o processamento até a rota
 - **@app.middleware("http"):** Define que o middleware intercepta requisições HTTP
 - **response = await call_next(request):** executa a rota e recebe a resposta dela



Exemplo Middleware para X-Process-Time

- Modelos pydantic para Requisição e Validação de Predição:

- ```
class PredictionRequest(BaseModel):
 message: str
```

```
class PredictionResponse(BaseModel):
 message: str
 intent: str
```

```
class PredictionValid(BaseModel):
 message: str
 valid: str
```



# Exemplo Middleware para X-Process-Time

- Rota para predição com contagem de tempo:

- ```
@app.post("/predict")
def predict(predictionRequest: PredictionRequest, response: fastapi.Response):
    start = time.time()
    predictionResponse = PredictionResponse(
        message=predictionRequest.message,
        intent='cancelamento'
    )
    final = time.time()
    response.headers['X-Process-Time'] = str(final - start)
    return predictionResponse
```

- Rota para validação de predição com contagem de tempo:

- ```
@app.post("/valid")
def valid(predictionRequest: PredictionRequest, response: fastapi.Response):
 start = time.time()
 predictionResponse = PredictionValid(
 message=predictionRequest.message,
 valid='Válido' if len(predictionRequest.message) > 5 else "Inválido"
)
 final = time.time()
 response.headers['X-Process-Time'] = str(final - start)
 return predictionResponse
```



# Exemplo Middleware para X-Process-Time

- Middleware com contagem de tempo:

- ```
@app.middleware("http")
async def process_time(request: fastapi.Request, call_next):
    start = time.time()
    response = await call_next(request)
    final = time.time()
    response.headers["X-Process-Time"] = str(final - start)
    return response
```

- Rotas de predição e validação limpas:

- ```
@app.post("/predict")
def predict(predictionRequest: PredictionRequest, response: fastapi.Response):
 predictionResponse = PredictionResponse(
 message=predictionRequest.message,
 intent='cancelamento'
)
 return predictionResponse

@app.post("/valid")
def valid(predictionRequest: PredictionRequest, response: fastapi.Response):
 predictionResponse = PredictionValid(
 message=predictionRequest.message,
 valid='Válido' if len(predictionRequest.message) > 5 else "Inválido"
)
 return predictionResponse
```

# Headers de segurança

- **HSTS (HTTP Strict Transport Security):** Usar HTTPS para acessar este domínio
  - **Exemplo recomendável:**

```
response.headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
```
- **X-Frame-Options:** Controla se a página pode ser colocada dentro de uma tag iframe HTML
  - **Exemplo recomendável:**

```
response.headers["X-Frame-Options"] = "DENY"
```
- **X-Content-Type-Options:** evitar MIME sniffing
  - **MIME sniffing:** navegador adivinhar tipo do arquivo e tentar executar
  - **Exemplo recomendável:**

```
response.headers["X-Content-Type-Options"] = "nosniff"
```



# Headers de segurança

- **Content-Security-Policy: quais fontes de conteúdo podem ser carregadas e executadas pela página**
  - `default-src 'self'` → padrão: só o próprio domínio
  - `script-src 'self'` → JavaScript só do próprio domínio
  - `style-src 'self'` → CSS só do próprio domínio
  - `img-src ...` → imagens do próprio domínio e de `images.exemplo.com`
  - **Exemplo recomendável (pode afetar swagger):**  
`response.headers["Content-Security-Policy"] = "default-src 'self'"`



# CORS



# Middleware pronto FastAPI

- **Middleware pronto FastAPI:** middleware já implementado e pronto para uso
  - **CORSMiddleware:** controla quais origens podem acessar sua API pelo navegador
  - **GZipMiddleware:** comprime respostas maiores para reduzir a quantidade de dados trafegados
  - **HTTPSRedirectMiddleware:** força o acesso por HTTPS
- **Adicionar middleware pronto:**
  - ```
app.add_middleware(  
    MiddlewareClass,  
    parametro1=valor1,  
    parametro2=valor2  
)
```



CORS

- **CORSMiddleware:** Controla quais origens podem acessar sua API pelo navegador
 - **Principais parâmetros:**
 - **allow_origins:** lista de origens que podem acessar sua API pelo navegador
 - **allow_methods:** lista de métodos HTTP permitidos para requisições CORS
 - **allow_headers:** lista de headers que o frontend pode enviar
 - **allow_credentials:** booleano indicando se navegador pode enviar credenciais em requisições entre origens, como cookies e headers de autenticação em certos cenários

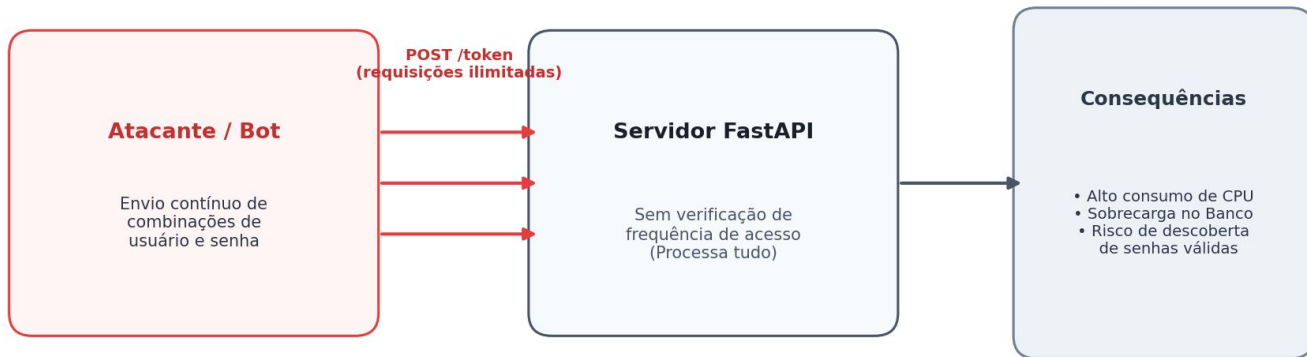


Brute Force e Rate Limiting



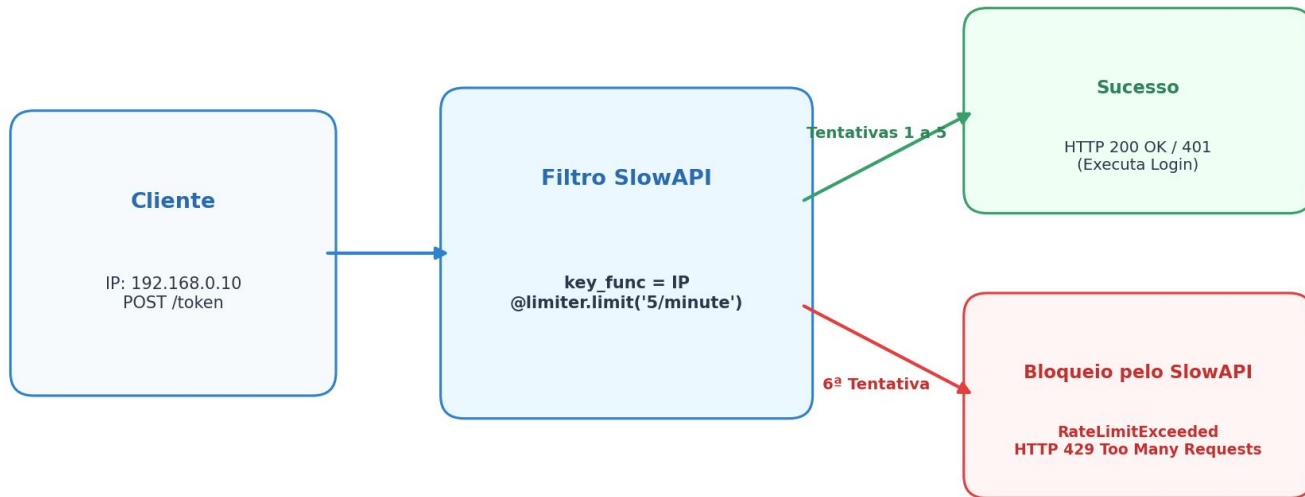
Brute Force Attack

- **Brute Force Attack:** atacante tenta descobrir uma senha tentando todas as combinações possíveis
 - Sem nenhum controle, um atacante pode enviar centenas ou milhares de tentativas
 - **Exemplo:**
 - `POST /token` com usuário e senha
 - `admin / 123`, `admin / admin`, `admin / password`,
`admin / 123456`, ...



Rate Limiting

- **Rate limiting:** técnica que limita quantas requisições um cliente pode fazer durante determinado período
 - **Exemplo:**
 - **Regra:** 5 requisições por minuto
 - **Execução:** 1ª req → permitida, 2ª req → permitida, 3ª req → permitida, 4ª req → permitida, 5ª req → permitida, **6ª req → bloqueada,**



SlowAPI

- **SlowAPI:** biblioteca de rate limiting utilizada com FastAPI e Starlette
 - **Permite definir limites diretamente nos endpoints**
- **Instalação:** `pip install slowapi`
- **Criação do limiter:**
 - **Imports:**
 - `from slowapi import Limiter`
 - `from slowapi.util import get_remote_address`
 - **Classe Limiter:**
 - `limiter = Limiter( key_func=... )`
 - `limiter`: objeto utilizado para limitar
 - `key_func`: define como os clientes serão identificados
 - `get_remote_address`: cliente é identificado pelo seu endereço IP
 - **Exemplo:**
 - `limiter = Limiter(key_func=get_remote_address)`



SlowAPI

- **Aplicação do limite:**
 - `@limiter.limit(str_limite)`
 - `str_limite`: string contendo código de restrição
 - **Exemplos:**
 - `"5/minute"`, `"10/minute"`, `"100/hour"`, `"1000/day"`
 - Decorador aplicado entre decorador de rota e definição de função de rota
 - Envolve a função do endpoint e verifica o limite antes de permitir sua execução
 - Não retorna algo que normalmente é utilizado diretamente no código
 - **Request obrigatório:**
 - `def func_route ( request: Request, ... ):`
 - SlowAPI necessita da request como parâmetro da rota

SlowAPI

- **Configuração no FastAPI:**
 - **Importar exceção:**
 - `from slowapi import _rate_limit_exceeded_handler`
 - **Importar tipo de erro:**
 - `from slowapi.errors import RateLimitExceeded`
 - Transforma a exceção em uma resposta HTTP apropriada
HTTP 429 Too Many Requests
 - **Configurar estado FastAPI:**
 - `api.state`: permite armazenar objetos relacionados à aplicação
 - `add_exception_handler(exception, handler)`:
 - `exception`: tipo de exceção a ser tratada
 - `handler`: Função executada quando essa exceção ocorrer
 - Registra uma função para tratar determinado tipo de exceção



Configuração de RateLimiting com SlowAPI

- Configuração do limitador na rota de predição:

- ```
from fastapi import Request
from slowapi import Limiter
from slowapi.util import get_remote_address

from fastapi import APIRouter, Depends, HTTPException
from sqlmodel import Session, select

from database import get_session
from models.prediction_sql_model import Prediction
from models.user_sql_model import User
from security.dependencies import get_current_user

router = APIRouter()
limiter = Limiter(key_func=get_remote_address)

@router.get("/predicitons")
@limiter.limit("5/minute")
def get_all_predicitons(
 request: Request,
 session: Session = Depends(get_session),
 ...
```

# Configuração de RateLimiting com SlowAPI

- Configuração do limitador na rota de predição:

```
from fastapi import Request
from slowapi import Limiter
from slowapi.util import get_remote_address
```

Imports

```
from fastapi import APIRouter, Depends, HTTPException
from sqlmodel import Session, select
```

```
from database import get_session
from models.prediction_sql_model import Prediction
from models.user_sql_model import User
from security.dependencies import get_current_user
```

```
router = APIRouter()
limiter = Limiter(key_func=get_remote_address)
```

Limitador

```
@router.get("/predicitons")
@limiter.limit("5/minute")
def get_all_predicitons(
 request: Request,
 session: Session = depends(get_session),
 ...
```

Configuração da  
rota

# Configuração de RateLimiting com SlowAPI

- Configuração do FastAPI:

- ```
from slowapi import _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded
from routes.prediction_route import limiter as prediction_limiter

from fastapi import FastAPI
from sqlmodel import SQLModel

from database import engine
from routes.prediction_route import router as prediction_router

app = FastAPI()
app.state.limiter = prediction_limiter
app.add_exception_handler(
    RateLimitExceeded,
    _rate_limit_exceeded_handler
)
...
```

Configuração de RateLimiting com SlowAPI

- Configuração do FastAPI:

```
from slowapi import _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded
from routes.prediction_route import limiter as prediction_limiter
```

Imports

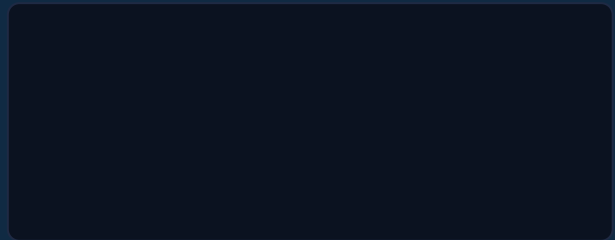
```
from fastapi import FastAPI
from sqlmodel import SQLModel
```

```
from database import engine
from routes.prediction_route import router as prediction_router
```

```
app = FastAPI()
app.state.limiter = prediction_limiter
app.add_exception_handler(
    RateLimitExceeded,
    _rate_limit_exceeded_handler
)
...
```

Tratamento de
exceção

Scan passivo OWASP ZAP



OWASP ZAP

- **OWASP ZAP (Zed Attack Proxy):** ferramenta gratuita e open source para testar a segurança de aplicações web e APIs
 - Proxy entre o cliente e a aplicação. Assim, consegue observar as requisições e respostas HTTP e procurar possíveis problemas de segurança
- **Download ZAP:** <https://www.zaproxy.org/download/>
- **Scan Passivo:** ZAP apenas observa o tráfego que já está acontecendo e analisa as mensagens
- **Scan Ativo:** Envia requisições próprias, tenta explorar possíveis falhas e Pode alterar comportamento/dados



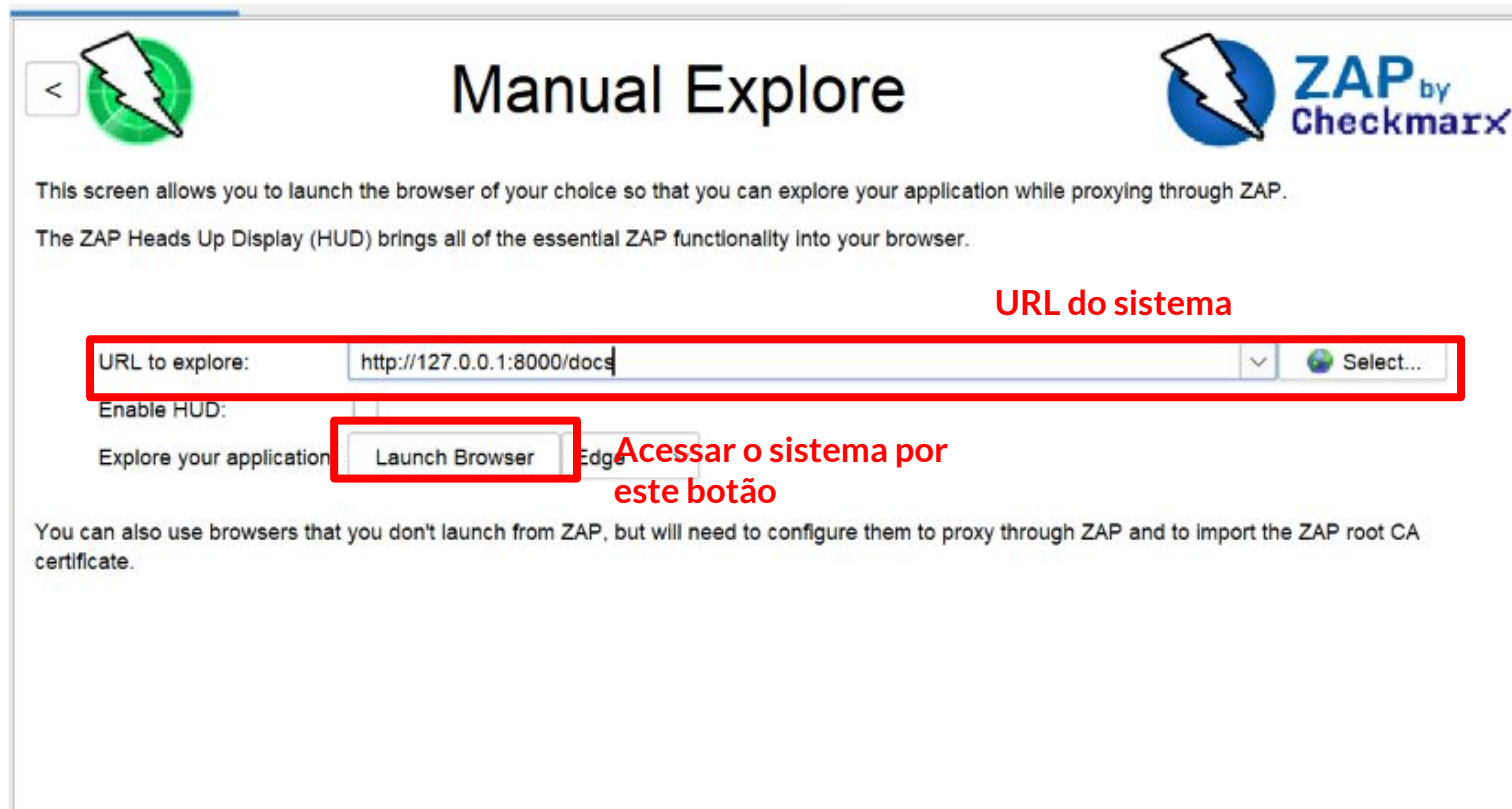
OWASP ZAP Scan Passivo

- **OWASP ZAP (Zed Attack Proxy):** ferramenta gratuita e open source para testar a segurança de aplicações web e APIs
 - Proxy entre o cliente e a aplicação. Assim, consegue observar as requisições e respostas HTTP e procurar possíveis problemas de segurança
- **Download ZAP:** <https://www.zaproxy.org/download/>
- **Scan Passivo:** ZAP apenas observa o tráfego que já está acontecendo e analisa as mensagens
- **Scan Ativo:** Envia requisições próprias, tenta explorar possíveis falhas e Pode alterar comportamento/dados

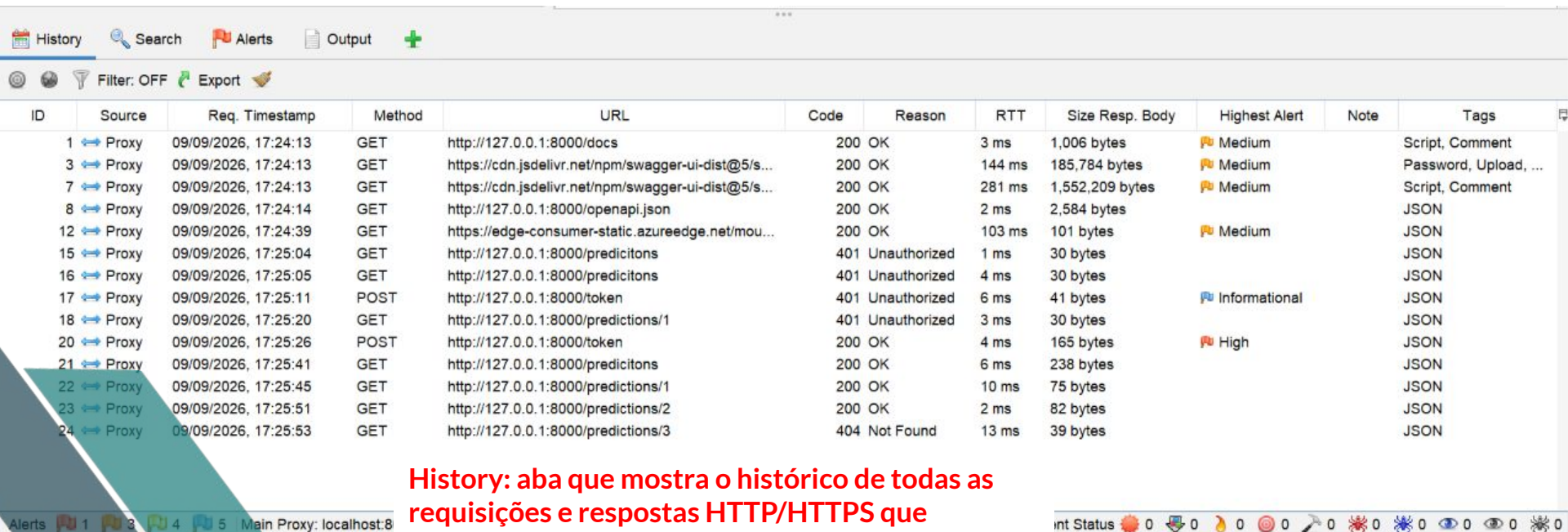




OWASP ZAP Scan Passivo



OWASP ZAP Scan Passivo



The screenshot displays the OWASP ZAP interface with the 'History' tab selected. The main area shows a table of captured HTTP traffic. The table has columns for ID, Source, Req. Timestamp, Method, URL, Code, Reason, RTT, Size Resp. Body, Highest Alert, Note, and Tags. The data shows various requests to a local application (127.0.0.1:8000) and external services like Swagger UI and Azure Edge. Alerts are shown in the bottom left, and a status bar is at the bottom right.

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
1	Proxy	09/09/2026, 17:24:13	GET	http://127.0.0.1:8000/docs	200	OK	3 ms	1,006 bytes	Medium		Script, Comment
3	Proxy	09/09/2026, 17:24:13	GET	https://cdn.jsdelivr.net/npm/swagger-ui-dist@5/s...	200	OK	144 ms	185,784 bytes	Medium		Password, Upload, ...
7	Proxy	09/09/2026, 17:24:13	GET	https://cdn.jsdelivr.net/npm/swagger-ui-dist@5/s...	200	OK	281 ms	1,552,209 bytes	Medium		Script, Comment
8	Proxy	09/09/2026, 17:24:14	GET	http://127.0.0.1:8000/openapi.json	200	OK	2 ms	2,584 bytes			JSON
12	Proxy	09/09/2026, 17:24:39	GET	https://edge-consumer-static.azureedge.net/mou...	200	OK	103 ms	101 bytes	Medium		JSON
15	Proxy	09/09/2026, 17:25:04	GET	http://127.0.0.1:8000/predicitions	401	Unauthorized	1 ms	30 bytes			JSON
16	Proxy	09/09/2026, 17:25:05	GET	http://127.0.0.1:8000/predicitions	401	Unauthorized	4 ms	30 bytes			JSON
17	Proxy	09/09/2026, 17:25:11	POST	http://127.0.0.1:8000/token	401	Unauthorized	6 ms	41 bytes	Informational		JSON
18	Proxy	09/09/2026, 17:25:20	GET	http://127.0.0.1:8000/predictions/1	401	Unauthorized	3 ms	30 bytes			JSON
20	Proxy	09/09/2026, 17:25:26	POST	http://127.0.0.1:8000/token	200	OK	4 ms	165 bytes	High		JSON
21	Proxy	09/09/2026, 17:25:41	GET	http://127.0.0.1:8000/predicitions	200	OK	6 ms	238 bytes			JSON
22	Proxy	09/09/2026, 17:25:45	GET	http://127.0.0.1:8000/predictions/1	200	OK	10 ms	75 bytes			JSON
23	Proxy	09/09/2026, 17:25:51	GET	http://127.0.0.1:8000/predictions/2	200	OK	2 ms	82 bytes			JSON
24	Proxy	09/09/2026, 17:25:53	GET	http://127.0.0.1:8000/predictions/3	404	Not Found	13 ms	39 bytes			JSON

History: aba que mostra o histórico de todas as requisições e respostas HTTP/HTTPS que passaram pelo ZAP, permitindo verificar quais endpoints foram acessados, métodos utilizados, códigos de resposta e detalhes completos das mensagens trocadas com a aplicação.

History

Search

Alerts

Output

Alerts (13)

> Authentication Credentials Captured

> Content Security Policy (CSP) Header Not Set

> GET:http://127.0.0.1:8000/docs

> Cross-Domain Misconfiguration (3)

> Sub Resource Integrity Attribute Missing (2)

> Cross-Domain JavaScript Source File Inclusion

> Strict-Transport-Security Header Not Set

> Timestamp Disclosure - Unix (Systemic)

> X-Content-Type-Options Header Missing

> Authentication Request Identified (2)

> Modern Web Application

> Re-examine Cache-control Directives

> Retrieved from Cache (2)

> Session Management Response Identified

Content Security Policy (CSP) Header Not Set

URL: http://127.0.0.1:8000/docs

Risk: Medium

Confidence: High

Parameter:

Attack:

Evidence:

CWE ID: 693

WASC ID: 15

Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)

Alert Reference: 10038-1

Input Vector:

Description:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered

Other Info:

Alerts: aba que apresenta os possíveis problemas de segurança identificados pelo ZAP durante a análise do tráfego, organizados por severidade, como High, Medium, Low e Informational, além de mostrar descrição, evidência, URL afetada e

Current Status 0 0 0 0 0 0 0 0 0 0

Documentação de descobertas

- **Finding:** nome do problema identificado pelo ZAP. Ex.: Content Security Policy (CSP) Header Not Set.
- **Severidade:** nível de risco atribuído pelo ZAP, normalmente Medium ou High.
- **Confiança:** indica o quanto o ZAP está confiante de que o problema realmente existe, por exemplo Low, Medium ou High.
- **URL afetada:** endpoint ou página onde o problema foi encontrado. Ex.: <http://127.0.0.1:8000/docs>.
- **O que foi detectado:** explique objetivamente o que está faltando ou configurado de forma insegura.
- **Por que é um problema:** descreva o risco de segurança causado por esse finding.
- **Correção realizada:** explique o que você alterou no código ou na configuração para resolver o problema.
- **Validação:** informe que executou novamente o scan para verificar se o finding desapareceu.
- **Risco aceito, se não corrigido:** caso você decida não corrigir, explique tecnicamente por que o risco foi aceito.

Documentação de descobertas

- **Exemplo documentação finding:**

- **Finding: Content Security Policy (CSP) Header Not Set**

Severidade: Medium

Confiança: High

URL afetada:

<http://127.0.0.1:8000/docs>

O que foi detectado:

O OWASP ZAP identificou que a resposta HTTP do endpoint `/docs` não possui o header `Content-Security-Policy` (CSP).

Por que é um problema:

Sem uma política CSP, o navegador não possui restrições explícitas sobre quais origens podem fornecer scripts, estilos, imagens e outros conteúdos. Isso reduz a proteção contra ataques como Cross-Site Scripting (XSS) e injeção de conteúdo malicioso.

Correção realizada:

Foi adicionado o header CSP no middleware da aplicação FastAPI:

```
response.headers["Content-Security-Policy"] = "default-src 'self'"
```

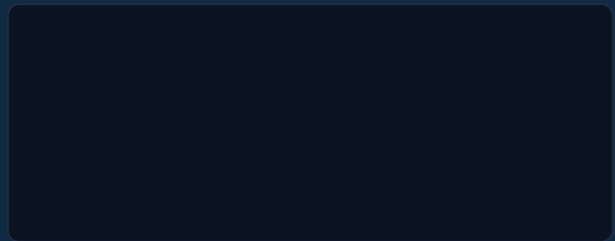
Com isso, por padrão, o navegador passa a permitir o carregamento de recursos somente da própria origem da aplicação.

Validação:

Após a correção, a API deve ser reiniciada e o scan passivo executado novamente no OWASP ZAP. O objetivo é verificar se o alerta `Content Security Policy (CSP) Header Not Set` deixa de aparecer.



EDA Completo + Teste de Hipótese



Análise Exploratória dos Dados

(variáveis textuais)

- **Passos do EDA:**

- Compreensão do problema e do dataset
- Inspeção inicial
- Verificação da qualidade dos dados
- Limpeza e preparação dos dados
- Análise Univariada
- Análise Multivariada
- Identificação de Outliers e Anomalias
- Documentação do EDA e conclusões



Limpeza e Preparação dos Dados

(variáveis textuais)

- **Principais operações:**

1. Remover/tratar registros duplicados desnecessários;
2. Remover/tratar valores ausentes tratados de forma justificada;
3. Corrigir tipos das colunas;
4. Corrigir espaços, erros de preenchimento e inconsistências evidentes;
5. Corrigir valores inválidos.
6. Remover caracteres de pontuação
7. Contagem de número de palavras e tamanho do texto
8. Vetorização de contagem de palavras



Análise Univariada (**variáveis textuais**)

- **Variáveis numéricas:**
 1. média, mediana, mínimo e máximo;
 2. quartis, variância e desvio-padrão;
 3. distribuição dos valores;
- **Variáveis categóricas:**
 1. quantidade de categorias;
 2. frequência absoluta e relativa;
 3. categoria mais frequente;
- **Variáveis textuais:**
 1. estatísticas tamanho do texto e quantidade palavras;
 2. distribuição tamanho do texto e quantidade de palavras;
 3. palavras mais frequentes;



Análise Multivariada (variáveis textuais)

- **Variáveis numéricas:**
 1. Correlação linear entre alvo x independentes
 2. Correlação linear entre variáveis independentes
 3. Correlação não-linear entre alvo x independentes
 4. Correlação não-linear entre variáveis independentes
 5. Analisar variáveis preditoras
 6. Analisar e tratar variáveis redundantes
- **Variáveis categóricas:**
 1. Associação entre alvo x independentes
 2. Associação entre variáveis independentes
 3. Analisar variáveis preditoras
 4. Analisar e tratar variáveis redundantes
- **Variáveis textuais:**
 - Associação entre variáveis do CountVectorizer x alvo
 - Associação entre variáveis do CountVectorizer
 - Analisar variáveis preditoras do CountVectorizer
 - Analisar e tratar variáveis redundantes do CountVectorizer



Identificar outliers e anomalias

- **Variáveis numéricas:**
 1. Selecionar método de identificação de outlier;
 1. Q-Q Plot, Skewness, Teste Shapiro-Wilk
 2. Identificar outliers com IQR ou ZScore
- **Variáveis categóricas:**
 1. Identificar categorias raras;
 2. Identificar possível desequilíbrio entre as classes.
- **Variáveis textuais:**
 1. Criar/identificar variáveis numéricas a partir da variável textual
 - 1.1. Selecionar método de identificação de outlier;
 - 1.1.1. Q-Q Plot, Skewness, Teste Shapiro-Wilk
 - 1.2. Identificar outliers com IQR ou ZScore
 2. Identificar termos raros do CountVectorizer



Teste de hipótese

- **Passos para teste de hipótese:**
 - Formular as hipóteses
 - H_0 : hipótese nula
 - H_1 : hipótese alternativa
 - Definir o nível de significância (geralmente, $\alpha = 0,05$)
 - Escolher teste estatístico (Teste t ou Mann-Whitney)
 - Aplicar o teste estatístico
 - Tomar a decisão
 - Se $p < \alpha \rightarrow$ rejeita H_0
 - Se $p \geq \alpha \rightarrow$ não rejeita H_0



Teste de hipótese

- **Exemplo:**
 - H_0 : Cancelamento e Atraso possuem a mesma média de palavras.
 - H_1 : Cancelamento e Atraso possuem médias de palavras diferentes.
- **Se teste retornar:** $p = 0,01$ e $\alpha = 0,05$
Então: $p < \alpha \rightarrow$ rejeita H_0
 - **Conclusão:** Existe evidência de diferença na quantidade média de palavras entre as duas intenções.
- **Se teste retornar:** $p = 0,30$ e $\alpha = 0,05$
Então: $p \geq \alpha \rightarrow$ não rejeita H_0
 - **Conclusão:** Não há evidência suficiente para afirmar que as médias são diferentes.



Teste de hipótese

- **Teste t e teste Mann-Whitney:** testes para verificar se **dois grupos apresentam valores diferentes para uma determinada variável**
 - **Teste t:**
 - Amostras independentes
 - Compara média de dois grupos
 - **Teste Mann-Whitney:**
 - distribuição muito assimétrica
 - muitos outliers
 - ausência de normalidade
 - valores discretos ou distribuições estranhas



Teste de hipótese

- **Exemplo dataset:**

texto	intenção	qtd_palavras
"quero cancelar meu pedido"	cancelamento	4
"meu produto ainda não chegou"	atraso	5
"cancele minha compra"	cancelamento	3
"gostaria de saber onde está meu pedido"	atraso	7

- **Objetivo:**

- Comparar cancelamento x atraso em relação a quantidade de palavras

Teste de hipótese

- **Teste t:** verifica se a média de uma variável numérica é diferente entre dois grupos
 - Exemplo:
 - Cancelamento: média = 6,2 palavras
 - Atraso: média = 9,8 palavras
- **Estatística t:**
 - t grande: grande diferença entre as médias
 - t pequeno: pequena diferença entre as médias
- **p-valor:**
 - **$p < 0,05$:** diferença estatisticamente significativa
 - **$p \geq 0,05$:** não há evidência suficiente de diferença
- **Teste t de Welch:** grupos podem ter variâncias diferentes



Teste de hipótese

- **Teste Mann-Whitney:** verifica se os valores de um grupo tendem a ser maiores ou menores que os valores do outro grupo
 - **Exemplo:**
 - Cancelamento: 3, 4, 5, 5, 6
 - Atraso: 7, 8, 9, 10, 12
 - Valores (dois grupos) ordenados: 3 4 5 5 6 7 8 9 10 12
 - Valor e ranks: 3 (1), 4 (2), 5 (3,5), 5 (3,5), 6 (5), 7 (6), 8 (7), 9 (8), 10 (9), 12 (10)
- **Estatística U:** a partir desses ranks, verifica se essa separação é grande demais para ser explicada pelo acaso
 - U próximo de 0: forte separação em uma direção.
 - U maior do que 0: grupos tendem a estar misturados
- **two-sided:** verificar qualquer diferença entre os grupos



Análise Exploratória de Dados (EDA)



Compreensão do problema e dataset

- **Domínio do problema:** definir claramente qual problema se deseja resolver e o que se espera como solução
- **Exemplos:**
 - detectar spam
 - prever doenças
 - reconhecer imagens
 - classificar intenções de clientes
- **Exemplo Domínio de Problema:**
 - *Classificar automaticamente a intenção de uma mensagem enviada por um cliente*
 - **Dataset:** Customer Support Intent
 - **Link:**
 - <https://www.kaggle.com/datasets/scodepy/customer-support-intent-dataset>



Compreensão do problema e dataset

- **Exemplos:**
 - **Mensagem:** "Gostaria de cancelar minha assinatura."
 - **Saída esperada:** Cancelamento
 - **Mensagem:** "Meu boleto venceu."
 - **Saída esperada:** Pagamento
 - **Mensagem:** "Esqueci minha senha."
 - **Saída esperada:** Suporte Técnico



Inspeção Inicial

- **O que deve ser inspecionado?**
 1. Tamanho do dataset
 2. Quais são as variáveis
 3. Significado colunas
 4. Tipos dos dados
 5. Organização dos registros
 6. Exemplos de valores
 7. Identificação de variáveis:
 1. Numéricas, categóricas, textuais ou temporais
 8. Variáveis relevantes para o objetivo da análise
 9. Possíveis problemas nos dados



Verificação da Qualidade dos Dados

- **O que deve ser verificado?**
 1. Valores ausentes e em qual quantidade;
 2. Registros duplicados;
 3. Tipos dos dados estão corretos;
 4. Categorias escritas de formas diferentes;
 5. Textos vazios ou espaços extras;
 6. Valores impossíveis ou fora do contexto;
 7. Erros de digitação ou preenchimento;
 8. Problemas que precisam ser corrigidos antes da análise.



Documentação do EDA e Conclusões

- **Documentação do EDA:** consiste na organização do notebook e das observações no markdown
- Conclusões:
 - **Principais achados:** mostrar as principais descobertas nos dados
 - Desbalanceamento, variáveis mais importantes, valores ausentes, outliers, padrões e possíveis redundâncias, etc.
 - **Operações realizadas:** mostrar o que foi feito no EDA nas fases anteriores
 - Limpeza, remoção de duplicatas, tratamento de valores ausentes, análises estatísticas, correlações, testes de associação e identificação de outliers, etc.



Tarefa em grupo

- **Realizar EDA com dataset Bank Marketing:**
 - O objetivo é prever se um cliente aceitará ou não contratar um depósito bancário, portanto também é um problema de classificação binária.
 - variáveis numéricas, como age, balance e duração do contato;
 - variáveis categóricas, como job, marital, education e housing;
 - valores desconhecidos em algumas categorias;
 - variável-alvo y, com valores yes e no
- **Link dataset Bank Marketing:**
<https://www.kaggle.com/dvaser/bank-marketing>



Aplicação FastAPI

Da instalação à primeira API

```
pip install fastapi  
fastapi dev main.py  
fast
```

Criar aplicação API RESTFUL com fastapi
com GET e POST

O que é uma API?

API — Application Programming Interface permite que diferentes aplicações se comuniquem.



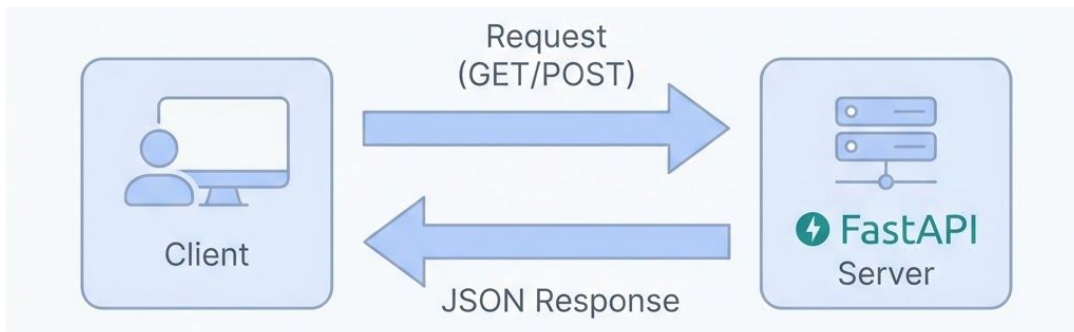
Frontend

- Frontend não precisa saber como os produtos estão armazenados.
- Ele apenas precisa saber como conversar com a API.



Cliente/Servidor

- Cliente solicita:
 - GET /produtos
- Servidor responde:
 - [{ "id": 1, "nome": "Notebook", "preco": 3500 }, { "id": 2, "nome": "Mouse", "preco": 100 }]



Métodos HTTP



GET

- Consultar dados
 - Exemplo: listar produtos



POST

- Criar dados
 - Exemplo: cadastrar produto



PUT

- Atualizar um recurso
 - Exemplo: alterar produto



DELETE

- Excluir um recurso
 - Exemplo: remover produto

Métodos HTTP



GET

- Consultar dados
 - Exemplo: listar produtos



POST

- Criar dados
 - Exemplo: cadastrar produto



PUT

- Atualizar um recurso
 - Exemplo: alterar produto



DELETE

- Excluir um recurso
 - Exemplo: remover produto

Primeira aplicação

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"mensagem": "Minha primeira API"}
```



Importando FastAPI

- Importando a classe principal do framework.



Criando a aplicação

- app representa a aplicação
- Nesse objeto são registrados os endpoints.



Criando uma rota

- - `@app.get("/")` informa ao FastAPI:
 - “Quando alguém fizer uma requisição GET para /, execute a função abaixo”
- Decorator associa uma rota e um método HTTP a uma função Python.



Criando a função

- ```
def home():
 return {"mensagem": "Minha primeira API"}
```

  - Quando o endpoint for acessado, a função será executada.
- `{"mensagem": "Minha primeira API"}`
  - Enviado ao cliente como uma resposta JSON.

# Executando a aplicação

---



## No terminal

- `fastapi dev main.py`



## No navegador

- `http://127.0.0.1:8000`



## Resposta no navegador

- `{ "mensagem": "Minha primeira API" }`



## Swagger

- <http://127.0.0.1:8000/docs> abre uma interface de acesso a API chamada swagger

# Criando um recurso: produtos

```
produtos = [
 {
 "id": 1,
 "nome": "Notebook",
 "preco": 3500.0
 },
 {
 "id": 2,
 "nome": "Mouse",
 "preco": 100.0
 }
]
```



## Produtos

- Essa lista funcionará temporariamente como nosso "banco de dados".
- Em uma aplicação real, provavelmente utilizaríamos um banco de dados

# Criando um endpoint para listar produtos

```
@app.get("/produtos")
def listar_produtos():
 return produtos
```



**GET /produtos ou <http://127.0.0.1:8000/produtos>**

- Resultado:
  - [
    - {
      - "id": 1,
      - "nome": "Notebook",
      - "preco": 3500.0
    - },
    - {
      - "id": 2,
      - "nome": "Mouse",
      - "preco": 100.0
    - }
  - ]

# Parâmetros de rota

```
@app.get("/produtos/{produto_id}")
def buscar_produto(produto_id: int):

 for produto in produtos:
 if produto["id"] == produto_id:
 return produto

 return {"erro": "Produto não encontrado"}
```



## GET /produtos/1 ou http://127.0.0.1:8000/produtos/1

- produto\_id: int indica que produto\_id deve ser um inteiro
- FastAPI reconhece {produto\_id} como parâmetro da rota e utiliza a anotação int para converter e validar o valor recebido
- Resultado:
  - {  
    "id": 1,  
    "nome": "Notebook",  
    "preco": 3500.0  
}

# Retornando erros HTTP

```
@app.get("/produtos/{produto_id}")
def buscar_produto(produto_id: int):

 for produto in produtos:
 if produto["id"] == produto_id:
 return produto

 raise HTTPException(
 status_code=404,
 detail="Produto não encontrado"
)
```



**GET /produtos/abc ou  
http://127.0.0.1:8000/produtos/abc**

- Produto abc não existe
- Agora, quando um produto não existir, a API retorna:
  - 404 Not Found
- Resultado:
  - { "detail": "Produto não encontrado" }

# Query parameters

```
@app.get("/listar_produtos")
def listar_produtos(limite: int = 10):
 return produtos[:limite]
```



## GET /produtos?limite=1

- Limite não aparece dentro da rota: "/produtos"
  - FastAPI interpreta esse argumento como um query parameter.
- Resultado:
  - [
    - {
      - "id": 1,
      - "nome": "Notebook",
      - "preco": 3500.0

# Criando dados com POST

```
@app.post("/produtos", status_code=201)
async def criar_produto(request: Request):
```

```
 produto = await request.json()
```

```
 novo_produto = {
 "id": len(produtos) + 1,
 "nome": produto["nome"],
 "preco": produto["preco"]
 }
 produtos.append(novo_produto)
```

```
 return novo_produto
```



## POST /produtos

- Cliente envia:
  - {  
    "nome": "Teclado",  
    "preco": 250  
}
- Cliente recebe:
  - {  
    "id": 3,  
    "nome": "Teclado",  
    "preco": 250  
}



## Método request.json()

- Lê o corpo da requisição e o converte para um dicionário Python.

# Testando o POST com requests

```
import requests

url1 =
"http://127.0.0.1:8000/criar_produto"
url2=
"http://127.0.0.1:8000/listar_produtos"
url3=
"http://127.0.0.1:8000/listar_produtos?lim
ite=4"
novo_produto = {
 "nome": "Teclado",
 "preco": 4000.0
}
response = requests.post(url1,
json=novo_produto)
response = requests.post(url1,
json=novo_produto)
response = requests.get(url3)
print(response.json())
```



## Instalar requests

- pip install requests



## requests.get e requests.post

- Realizam requisições GET e POST HTTP
  - request.get: apenas url
  - request.post: url e conteúdo no formato json a ser enviado



## Outro Terminal

- python test.py
  - Se o arquivo se chamar test.py

# Modelos Pydantic

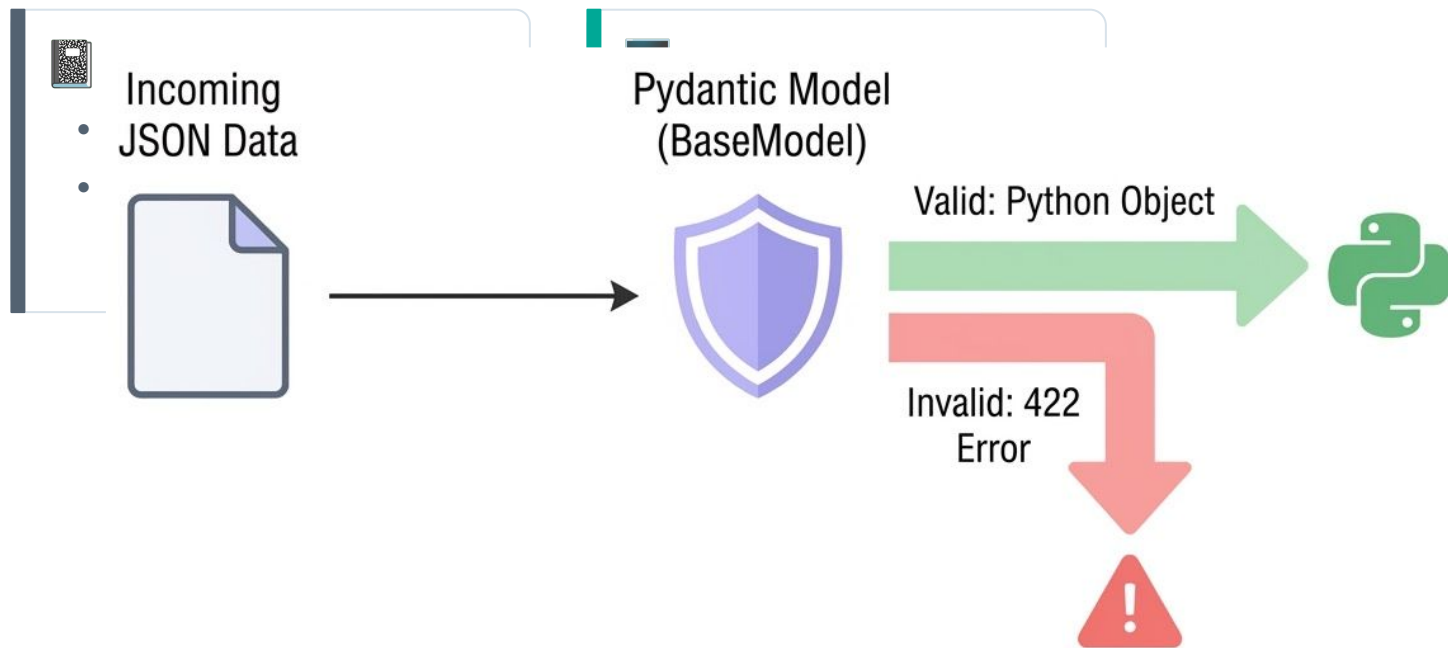
Da instalação à primeira API

```
pip install fastapi
fastapi dev main.py
fast
```

Criar aplicação API RESTFUL com fastapi  
com GET e POST

# Modelos Pydantic

**Pydantic: biblioteca Python utilizada para trabalhar com dados estruturados e validação baseada em tipos.**



# Modelo Pydantic



## Definição do modelo

- ```
class Produto(BaseModel):  
    nome: str  
    preco: float
```



Atributos esperados

- Um Produto possui:
 - nome → str
 - preco → float

Importando BaseModel

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Produto(BaseModel):
    nome: str
    preco: float
```



Produto

- Classe chamada Produto que herda BaseModel
- Formato esperado dos dados:
 - nome: str
 - preco: float
- Produto válido:
 - {
 "nome": "Notebook",
 "preco": 3500
}
- Produto inválido:
 - {
 "nome": 1000,
 "preco": 3500
}

Criando objetos Pydantic em Python

```
from pydantic import BaseModel
```

```
class Produto(BaseModel):  
    nome: str  
    preco: float
```

```
produto = Produto(  
    nome="Notebook",  
    preco=3500  
)
```

```
dados = produto.model_dump()  
  
print(dados)
```



Objeto Pydantic Produto

- Acesso aos seus atributos:
 - `print(produto.nome)`
`print(produto.preco)`



`produto.model_dump()`

- Converte um modelo para dicionário
- Resultado:
 - `{`
 `"nome": "Notebook",`
 `"preco": 3500.0`
 `}`

Utilizando o modelo no FastAPI

```
@app.post("/produtos", status_code=201)
def criar_produto(produto: Produto):

    novo_produto = {
        "id": len(produtos) + 1,
        "nome": produto.nome,
        "preco": produto.preco
    }

    produtos.append(novo_produto)

    return novo_produto
```



produto: Produto

- FastAPI percebe que Produto é um modelo Pydantic
 - Dados devem vir no corpo da requisição e devem possuir a estrutura definida por esse modelo.
- FastAPI realiza esse trabalho automaticamente.

Validação automática



Validação do modelo

- Classe:
 - `class Produto(BaseModel):`
 `nome: str`
 `preco: float`
- Envio 1:
 - `{`
 `"nome": "Teclado",`
 `"preco": "abc"`
 `}`
- Envio 2:
 - `{`
 `"nome": "Teclado"`
 `}`



Erro retornado

- Envio1:
 - O valor "abc" não pode ser utilizado como um número float
- Envio 2:
 - Os dois campos são obrigatórios
- FastAPI rejeitará automaticamente a requisição antes mesmo da execução da nossa função.

Adicionando um novo campo

```
class Produto(BaseModel):  
    nome: str  
    preco: float  
    categoria: str
```



Corpo esperado

- {
 "nome": "Teclado",
 "preco": 250,
 "categoria": "Periféricos"
 }
- Alterar modelo já modifica o formato esperado pela API

Campos opcionais e valores padrão

```
class Produto(BaseModel):  
    nome: str  
    preco: float  
    categoria: str  
    descricao: str | None = None  
    disponivel : bool = True
```

JSONs válidos

- {
 "nome": "Teclado",
 "preco": 250,
 "categoria": "Periféricos"
}
- {
 "nome": "Teclado",
 "preco": 250,
 "categoria": "Periféricos",
 "descricao": "Teclado mecânico"
}



Campos opcionais

- | None = None indica que o campo é opcional
- Opcionais:
 - descricao



Campos padrão

- var : type = value indica que value é padrão para var
- Valor padrão é opcional
- Padrão:
 - disponivel = True

Validações com Field

```
from pydantic import BaseModel, Field
class Produto(BaseModel):
    nome: str = Field(min_length=2)
    preco: float = Field(gt=0 )
    categoria: str
    descricao: str | None = None
    disponivel : bool = True
```



Validações de campos

- Field(gt=0) significa greater than 0 ou maior que zero
- Field(min_length=2) significa que o nome precisa ter pelo menos dois caracteres

Dica: Validações com Fields

<https://pydantic.dev/docs/validation/latest/concepts/fields/>

FASTAPI

Autenticação com OAuth2PasswordBearer

Da instalação à primeira API

```
pip install fastapi
fastapi dev main.py
fast
```

Criar aplicação API RESTFUL com
fastapi com GET e POST

Importando OAuth2PasswordBearer

```
from fastapi import
    FastAPI, HTTPException, Depends
from fastapi.security import
    OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme =
OAuth2PasswordBearer(tokenUrl="token")
...
```



oauth2_scheme =
OAuth2PasswordBearer(tokenUrl="token")

- Esquema de autenticação OAuth2 utilizando **Bearer Token**
- Quando utilizarmos esse objeto como uma dependência, FastAPI poderá procurar o token enviado no cabeçalho:
 - **Authorization: Bearer token-fake-admin**
- O parâmetro tokenUrl="token" indica qual endpoint será utilizado pelo cliente para obter o token
- OAuth2PasswordBearer(tokenUrl="token") não cria automaticamente a rota /token.
 - O tokenUrl informa ao esquema OAuth2 e à documentação da API onde o cliente poderá solicitar o token

Criando usuário fake

```
from fastapi import
    FastAPI, HTTPException, Depends
from fastapi.security import
    OAuth2PasswordBearer
from pydantic import BaseModel

app = FastAPI()

oauth2_scheme =
    OAuth2PasswordBearer(tokenUrl="token")

USUARIO_FAKE = {
    "username": "admin",
    "password": "1234"
}

TOKEN_FAKE = "token-fake-admin"
```



Segurança

- Armazenar senha dessa forma é propositalmente inseguro e está sendo feito apenas para que possamos enxergar o fluxo da autenticação sem introduzir hashing ou JWT

Criando a rota de login

```
from fastapi import FastAPI, HTTPException, Request, Depends
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from pydantic import BaseModel
from produto import Produto

app = FastAPI()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")

USUARIO_FAKE = {"username": "admin", "password": "1234"}
TOKEN_FAKE = "token-fake-admin"

@app.post("/token")
def login(form_data: OAuth2PasswordRequestForm =
Depends(OAuth2PasswordRequestForm)):
    if form_data.username != USUARIO_FAKE["username"] or \
        form_data.password != USUARIO_FAKE["password"]:
        raise HTTPException(status_code=401, detail="Usuário ou
senha inválidos.")
    return {
        "access_token": TOKEN_FAKE,
        "token_type": "Bearer"
    }

...
// Demais rotas
```

form_data: OAuth2PasswordRequestForm
=

Depends(OAuth2PasswordRequestForm)

- FastAPI recebe os dados enviados pelo cliente e disponibiliza:
 - form_data.username
 - form_data.password

var: class = Depends(method)

- Antes de executar a função login, execute a lógica representada por method e coloque o resultado na variável var.

Testando o token

```
@app.get("/teste-token")
def teste_token(token: str = Depends(oauth2_scheme)):
    return {"token_recebido": token}
```



Rota teste-token

- Cliente envia:
 - Authorization: Bearer t
- FastAPI utiliza oauth2_scheme para obter o conteúdo do token
- teste_token recebe
 - token = "token-fake-admin"
- teste-token envia:
 - {
 "token_recebido": "token-fake-admin"
}



Botão Authorize no Swagger

- Swagger cria botão "Authorize" para entrar com username e password

Função de validação de token

```
def validar_token(token: str = Depends(oauth2_scheme)):  
  
    if token != TOKEN_FAKE:  
        raise HTTPException(  
            status_code=401,  
            detail="Token inválido",  
            headers={"WWW-Authenticate": "Bearer"}  
        )  
  
    return token
```



validar_token

- Função para comparar o token recebido pelo usuário com o token real

Protegendo uma rota

```
@app.get("/listar_produtos_autenticado/")
def listar_produtos(limite: int = 10, token: str =
Depends(validar_token)):
    return produtos[:limite]
```



token: str = Depends(validar_token)

- Executa a função `listar_produtos_autenticado` somente após `validar_token`

Segurança com JWT

Da instalação à primeira API

```
pip install fastapi  
fastapi dev main.py  
fast
```

Criar aplicação API RESTFUL com
fastapi com GET e POST

JWT (JSON Web Token)



JWT (JSON Web Token)

- representação compacta de informações, chamadas claims, que pode ser assinada para permitir a verificação de sua integridade



Biblioteca PyJWT

- permite codificar e decodificar JSON Web Tokens em Python



Dinâmico

- O valor real será maior e poderá mudar a cada login dependendo dos dados utilizados para criá-lo.



Verificação

- Ao invés:
 - token-fake-admin
- Tem-se:
 - eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbiIsImV4cCI6MTc... .abc123...



instalação

- `py -m pip install pyjwt`

Importando PyJWT

```
from datetime import datetime, timedelta, timezone
import jwt
from jwt.exceptions import InvalidTokenError

from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel
```



Imports

- `datetime`: usado para registrar o momento de criação ou o limite de validade do token.
- `timedelta`: usado para somar minutos ou dias à data atual (ex: definir que o token dura 30 minutos).
- `timezone`: Gerencia fusos horários, garantindo que o cálculo de expiração do token seja preciso independentemente de onde o servidor esteja hospedado (usando o padrão UTC).
- `InvalidTokenError`: É o aviso de erro disparado automaticamente pelo jwt caso o sistema tente ler um token que foi falsificado ou adulterado.

Criando uma função gerar_token()

```
def gerar_token(username: str):  
  
    expiracao = datetime.now(timezone.utc) +  
    timedelta(minutes=30)  
  
    dados = {  
        "sub": username,  
        "exp": expiracao  
    }  
  
    token = jwt.encode(  
        dados,  
        SECRET_KEY,  
        algorithm=HS256  
    )  
  
    return token
```



função gerar_token

- A função recebe username
- expiracao é o momento de expiração do token
 - expiração = agora + 30 minutos
- sub (subject): identifica quem é o dono do token (geralmente o username ou ID do usuário)
- exp (expiration): identifica quando o token expira



função jwt.encode(dados, chave, algoritmo)

- dados: o conteúdo do token (payload), tipo sub e exp. Não é secreto, só codificado.
- SECRET_KEY: a chave usada pra assinar o token, garantindo que ninguém adultere o conteúdo sem ser detectado. Fica só no servidor.
- algorithm: qual algoritmo criptográfico gera essa assinatura (ex: "HS256").

Nova rota com JWT

```
@app.post("/token_jwt")
def login(form_data: OAuth2PasswordRequestForm =
Depends(OAuth2PasswordRequestForm )):

    if (
        form_data.username != USUARIO_FAKE["username"]
        or form_data.password !=
USUARIO_FAKE["password"]
    ):
        raise HTTPException(
            status_code=401,
            detail="Usuário ou senha inválidos"
        )

    token = gerar_token(form_data.username)

    return {
        "access_token": token,
        "token_type": "bearer"
    }
```

Função validar_token_jwt()

```
def validar_token_jwt(token: str = Depends(oauth2_scheme)):  
  
    erro = HTTPException(  
        status_code=401,  
        detail="Token inválido ou expirado",  
        headers={"WWW-Authenticate": "Bearer"}  
    )  
  
    try:  
  
        payload = jwt.decode(  
            token,  
            SECRET_KEY,  
            algorithms=["HS256"]  
        )  
  
        username = payload.get("sub")  
  
        if username is None:  
            raise erro  
  
    except InvalidTokenError:  
        raise erro  
  
    return username
```